

Voice AI & Agentic Systems

Building real-time conversational AI and autonomous multi-step agents

What This Module Is About

This module covers two powerful AI system patterns. Voice AI brings LLMs into real-time audio conversation — the 500ms latency target is the difference between feeling natural and feeling broken. Agentic systems go beyond single responses: they plan, use tools, execute multi-step workflows, and operate with a degree of autonomy.

Voice AI Pipeline

A real-time voice AI pipeline has seven stages, each contributing to latency:

Stage	Component	Latency Target
Audio Input	Microphone stream	—
STT	Speech-to-Text	< 100ms
Partial Transcripts	Interim results while speaking	Real-time
LLM Reasoning	Claude / GPT-4	< 200ms (streaming)
Token Streaming	Partial token output	Real-time
TTS	Text-to-Speech	< 80ms
Playback	Audio output	Real-time

Agentic System Concepts

- **Agent Memory** — working memory (current task), episodic memory (session history), semantic memory (long-term knowledge). Design which your agent needs.
- **Tool Use** — agents call external tools (search, calculator, database, APIs). Each tool must have a clear schema: name, description, input params, output format.
- **Single vs Multi-Agent** — single agents are simpler; multi-agent systems run sub-tasks in parallel for speed and specialisation.
- **Interruption Handling** — in voice AI, users interrupt mid-sentence. Your system must detect this and cancel the current TTS stream gracefully.
- **Chain-of-Thought Planning** — for complex tasks, prompt the agent to decompose the goal into sub-steps before acting. This dramatically improves reliability.

Tip: Start with batch (non-streaming) voice AI to get the pipeline working. Only add streaming once each stage (STT → LLM → TTS) is confirmed to work independently.

Resources for Deeper Learning

1 Deepgram — Real-Time Transcription Docs

<https://developers.deepgram.com/docs/getting-started-with-live-streaming-audio>

Deepgram has one of the fastest and most accurate STT APIs. Their live-streaming guide shows how to get real-time transcripts over WebSocket — essential for voice AI.

2 ElevenLabs API Documentation

<https://elevenlabs.io/docs/api-reference/getting-started>

ElevenLabs produces the most natural-sounding TTS available. Their streaming endpoint lets you start playing audio before the full response is generated.

3 Anthropic Tool Use Guide

<https://docs.anthropic.com/en/docs/build-with-claude/tool-use>

Official Claude documentation on tool use (function calling). Read this before building any agent — it covers tool schemas, parallel tool calls, and best practices.

4 LangGraph — Agent Orchestration

<https://langchain-ai.github.io/langgraph/>

LangGraph provides a graph-based framework for building stateful, multi-step agents. Useful for understanding agent control flow even if you build from scratch.

5 WebSocket Protocol — MDN

https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

Voice AI relies on WebSocket for real-time audio streaming. Understand the protocol, connection lifecycle, and error handling before your implementation.

6 OpenAI Realtime API Concepts

<https://platform.openai.com/docs/guides/realtime>

OpenAI's realtime API documentation has excellent conceptual diagrams of voice AI pipelines, latency considerations, and turn-taking — applicable to any voice system.

Module 4 Assignment — Build a Voice Interface or Simple Agent

- **OPTION A (Voice):** Build a batch voice pipeline — record a 5-second audio clip, send to Deepgram for transcription, send transcript to Claude, convert response to speech with ElevenLabs, play it back.
- **OPTION B (Agent):** Build a simple tool-using agent with Claude. Give it 2 tools: a 'search_web' stub and a 'calculator'. Ask it a multi-step question and verify it calls the right tools in the right order.
- **Whichever option you chose:** add error handling for every external API call. If STT fails, TTS fails, or the LLM times out, your system should recover gracefully with a user-friendly message.
- **Log latency for each stage separately** (STT ms, LLM ms, TTS ms). Identify the slowest stage and research one way to speed it up.
- **Add a brief architecture diagram of your pipeline to your GitHub README.**

Note: Either option builds directly towards your capstone. Pick the one that aligns with your project idea.